

zkASM: A Zero-Knowledge Assembly Language

Technical Report on the go-corset Implementation

Generated from go-corset source analysis

January 28, 2026

Abstract

This technical report provides a comprehensive analysis of zkASM, a zero-knowledge assembly language implemented within the go-corset compiler framework. zkASM serves as an intermediate representation that bridges high-level constraint specifications with low-level arithmetic circuits suitable for zero-knowledge proof systems. We examine the language syntax, compilation pipeline, constraint generation mechanisms, and field-agnostic execution model. The report details the novel register splitting algorithm for field independence, the vectorization optimization pass, and the bus-based inter-function communication protocol.

Contents

1 Introduction

1.1 Background and Motivation

Zero-knowledge proof systems require arithmetic constraints over finite fields to be expressed in a specific format amenable to cryptographic protocols. The go-corset project provides a compilation framework that transforms human-readable constraint specifications into optimized arithmetic intermediate representations (AIR).

zkASM (Zero-Knowledge Assembly) is a low-level programming language within this framework that provides:

- Explicit control over register allocation and data flow
- Direct mapping to arithmetic constraints
- Field-agnostic programming with automatic adaptation
- Efficient function composition via I/O buses

1.2 Document Organization

This report is organized as follows:

- Section 2 presents the zkASM language syntax and semantics
- Section 3 details the compilation pipeline architecture
- Section 4 explains the constraint generation process
- Section 5 describes the field-agnostic execution model
- Section 6 covers the executor and trace generation
- Section 7 discusses integration with the broader constraint system
- Section 8 provides implementation details and algorithms

2 zkASM Language Specification

2.1 Lexical Structure

The zkASM lexer recognizes the following token categories:

Category	Tokens	Examples
Keywords	fn, pub, const, var, if, goto, return	
Types	u1, u8, u16, u32, u128, u256	
Operators	+, -, *, /, ==, !=, <, >	
Delimiters	(,), {, }, ,, :	
Literals	Numbers (decimal, hex, binary)	42, 0xFF, 0b1010
Comments	Line comments	;; comment

Table 1: zkASM Token Categories

2.2 Type System

zkASM uses unsigned integer types with explicit bit widths:

$$\text{Type} ::= un \quad \text{where } n \in \{1, 8, 16, 32, 64, 128, 256\} \quad (1)$$

Each type un represents values in the range $[0, 2^n - 1]$. The type system enforces:

- Arithmetic operations preserve or extend bit widths
- Carry/overflow bits are explicitly captured
- Division produces quotient, remainder, and witness values

2.3 Function Declarations

Functions are declared with explicit input/output signatures:

```

1 fn function_name(param1 type1, param2=default type2)
2   -> (out1 type3, out2=padding type4) {
3   ;; function body
4   return
5 }
```

Listing 1: Function Declaration Syntax

Key features:

- **Default values:** Parameters can specify default values for padding
- **Multiple outputs:** Functions return multiple values with explicit types
- **Padding values:** Output padding values are used for trace expansion

2.4 Register Declarations

Internal registers are declared with the `var` keyword:

```

1 var counter u32           ;; Single register
2 var flag1, flag2 u1       ;; Multiple registers of same type
3 var temp u16              ;; Temporary computation register
```

Listing 2: Register Declarations

2.5 Instruction Set

2.5.1 Assignment Instructions

Assignments compute expressions and store results:

```

1 result = input1 + input2           ;; Simple addition
2 carry, sum = a + b                 ;; Addition with carry capture
3 q, r, w, s, ws = dividend / divisor ;; Division with full outputs
```

Listing 3: Assignment Examples

The assignment semantics for addition with carry:

$$\text{carry} \cdot 2^n + \text{sum} = a + b \quad (2)$$

where n is the bit width of `sum`.

2.5.2 Control Flow Instructions

```

1 ;; Conditional branch
2 if counter == 0 goto exit_label
3
4 ;; Unconditional branch
5 goto loop_start
6
7 ;; Labels
8 loop_start:
9     ;; instructions
10 exit_label:
11     return

```

Listing 4: Control Flow Examples

Supported comparison operators: ==, !=, <, <=, >, >=

2.5.3 Function Calls

Inter-function communication uses I/O buses:

```

1 ;; Call another function
2 result = other_function(arg1, arg2)
3
4 ;; Multiple return values
5 quot, rem = divide(dividend, divisor)

```

Listing 5: Function Call Example

2.5.4 Ternary Expressions

Conditional expressions for compact logic:

```

1 result = condition == 1 ? true_value : false_value

```

Listing 6: Ternary Expression

2.6 Complete Example

```

1 fn div(x u16, y=1 u16) -> (q u16, r u16) {
2     var sum u32
3     var wit u16
4     var witsum u17
5
6     ;; Compute division with witness
7     q, r, wit, sum, witsum = x / y
8     return
9 }

```

Listing 7: Division Function Implementation

The division operation generates constraints:

$$\text{sum} = q \cdot y + r \quad (3)$$

$$\text{witsum} = r + \text{wit} + 1 \quad (4)$$

$$\text{sum} = x \quad (5)$$

$$\text{witsum} = y \quad (6)$$

These constraints ensure $q = \lfloor x/y \rfloor$ and $r = x \bmod y$ with $r < y$.

3 Compilation Pipeline

3.1 Pipeline Overview

The zkASM compilation pipeline transforms source code through multiple intermediate representations:

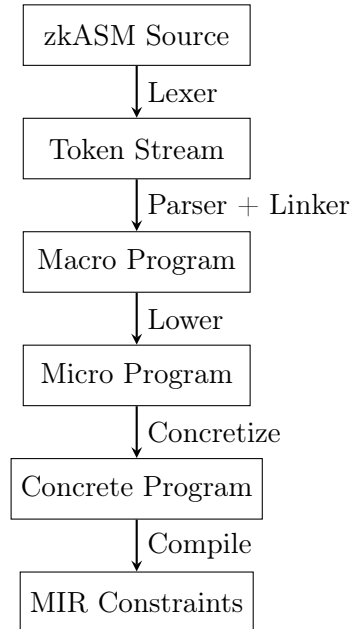


Figure 1: zkASM Compilation Pipeline

3.2 Lexical Analysis

The lexer (`assembler/lexer.go`) tokenizes source files into a stream of tokens. Key features:

- Support for multiple number formats (decimal, hexadecimal `0x`, binary `0b`)
- Underscore separators in numeric literals for readability
- Line comments with `;;`
- Source location tracking for error reporting

3.3 Parsing and Linking

The parser (`assembler/parser.go`) constructs an abstract syntax tree representing:

- Function declarations with register specifications
- Instruction sequences with label resolution
- Expression trees for computations

The linker (`assembler/linker.go`) performs:

- **Bus Resolution:** Maps function names to unique bus identifiers
- **Constant Substitution:** Replaces named constants with values
- **Cross-file References:** Resolves inter-file function calls

3.4 Validation

The validator (`assembler/validation.go`) performs two-phase checking:

Phase 1 - Instruction Validation:

- Register bit-width balance verification
- Signed arithmetic carry bit validation
- Field width requirement checking

Phase 2 - Control Flow Analysis:

- Reachability analysis (all paths reach `return`)
- Uninitialized register detection
- Label definition verification
- Input register write prevention

3.5 Macro to Micro Lowering

The lowering phase transforms high-level macro instructions into primitive micro instructions:

Macro Instruction	Micro Expansion
Assign	Assign + Jmp
IfGoto	Skip + Jmp + Jmp
Division	Division + Assign $\times 2$ + Skip $\times 2$ + Jmp + Fail
Call	InOut + Jmp
Return	Ret
Goto	Jmp

Table 2: Macro to Micro Instruction Mapping

3.6 Vectorization Optimization

The vectorization pass merges consecutive non-conflicting instructions:

Algorithm 1 Instruction Vectorization

```

1: function VECTORIZE(instructions)
2:   for each instruction  $i$  at PC  $p$  do
3:      $conflicts \leftarrow \emptyset$ 
4:      $merged \leftarrow \{i\}$ 
5:     for each following instruction  $j$  at PC  $p + k$  do
6:       if WritesConflict( $merged, j$ ) then
7:         break
8:       end if
9:        $merged \leftarrow merged \cup \{j\}$ 
10:    end for
11:     $instructions[p] \leftarrow MergeInstructions(merged)$ 
12:  end for
13:  return PRUNEUNREACHABLE(instructions)
14: end function

```

Conflict detection prevents merging when:

- Multiple instructions write to the same register
- Branch targets land in the middle of a merged instruction

4 Constraint Generation

4.1 Constraint Model

Each micro instruction generates polynomial constraints over finite field $\mathbb{F}$:

[Vanishing Constraint] A vanishing constraint is a polynomial $p(x_1, \dots, x_n) \in \mathbb{F}[x_1, \dots, x_n]$ that must evaluate to zero for all valid trace rows:

$$\forall i \in \text{Rows} : p(\text{trace}[i]) = 0 \quad (7)$$

4.2 Assignment Constraints

For an assignment $\mathbf{t}_n, \dots, \mathbf{t}_0 = \mathbf{expr}$:

$$\sum_{i=0}^n t_i \cdot 2^{w_i} = \text{eval}(\mathbf{expr}) \quad (8)$$

where $w_i = \sum_{j=0}^{i-1} \text{width}(t_j)$ is the bit offset of register t_i .

Example: For `carry`, `sum = a + b` with 8-bit registers:

$$256 \cdot \text{carry} + \text{sum} = a + b \quad (9)$$

4.3 Division Constraints

Division generates multiple verification constraints:

$$\text{sum} = q \cdot y + r \quad (10)$$

$$\text{witsum} = r + w + 1 \quad (11)$$

$$\text{sum} = x \quad (12)$$

$$\text{witsum} = y \quad (13)$$

Equations (??) and (??) ensure correct quotient/remainder. Equations (??) and (??) ensure $r < y$ via the witness $w = y - r - 1 \geq 0$.

4.4 Conditional Constraints

For conditional branches, constraints are guarded by the program counter:

$$(\text{PC}[i] = p) \Rightarrow \text{constraint}_p \quad (14)$$

This is encoded as:

$$(\text{PC}[i] - p) \cdot \text{constraint}_p^{-1} + \text{constraint}_p = 0 \quad (15)$$

or using indicator polynomials for mutual exclusivity.

4.5 Program Counter Constraints

For multi-line functions, the program counter evolution is constrained:

- **Sequential:** $\text{PC}[i + 1] = \text{PC}[i] + 1$
- **Jump:** $\text{PC}[i + 1] = \text{target}$
- **Return:** $\text{RET}[i] = 1$

4.6 Bus Constraints

Function calls use I/O buses with address and data lines:

$$\text{addr}[i] = \text{hash}(\text{inputs}) \quad (16)$$

$$\text{data}[i] = \text{outputs} \quad (17)$$

Bus constraints ensure that every call instance matches a valid function execution trace.

5 Field-Agnostic Execution

5.1 The Field Independence Problem

zkASM programs must execute correctly across different finite fields with varying characteristics:

Field	Modulus Bits	Max Register Width
BLS12-377	253	252
KoalaBear	31	30
GF(8209)	13	12

Table 3: Supported Fields and Their Constraints

A 256-bit register cannot be directly represented in any of these fields.

5.2 Register Splitting Algorithm

The solution is to split wide registers into multiple “limbs” that fit within the field:

[Register Splitting] For a register r of width w and field maximum width m , define:

$$r = \sum_{i=0}^{\lceil w/m \rceil - 1} r_i \cdot 2^{i \cdot m} \quad (18)$$

where each r_i has width $\min(m, w - i \cdot m)$.

5.3 Constraint Transformation

Original constraint for 16-bit addition with 8-bit field limit:

Before splitting:

$$\text{carry} \cdot 2^{16} + \text{sum} = a + b \quad (19)$$

After splitting (each register $\rightarrow$ 2 limbs):

$$c_0 \cdot 2^8 + \text{sum}_0 = a_0 + b_0 \quad (20)$$

$$\text{carry} \cdot 2^8 + \text{sum}_1 = a_1 + b_1 + c_0 \quad (21)$$

where c_0 is an internal carry between limbs.

Algorithm 2 Register Splitting Environment

```

1: function SPLITREGISTER(register  $r$ , maxWidth  $m$ )
2:    $w \leftarrow \text{width}(r)$ 
3:    $n \leftarrow \lceil w/m \rceil$ 
4:    $\text{limbs} \leftarrow []$ 
5:   for  $i = 0$  to  $n - 1$  do
6:      $\text{limbWidth} \leftarrow \min(m, w - i \cdot m)$ 
7:      $\text{limbs.append}(\text{NewRegister}(\text{limbWidth}))$ 
8:   end for
9:   return  $\text{limbs}$ 
10: end function

```

5.4 Limbs Map

The LimbsMap data structure tracks the relationship between original and split registers:

```

1 type LimbsMap struct {
2   maxWidth      uint
3   regsBefore    [] Register
4   regsAfter     [] Register
5   regMap        [] uint // original -> first limb index
6 }

```

Listing 8: LimbsMap Structure

6 Executor and Trace Generation

6.1 Execution Model

The executor simulates zkASM programs to generate execution traces:

```

1 type State struct {
2   pc          uint           // Program counter
3   terminal     bool          // Termination flag
4   state        [] big.Int    // Register values
5   registers    [] Register    // Register metadata
6   io           Map           // I/O subsystem for calls
7 }

```

Listing 9: Executor State

6.2 Instruction Execution

Each micro instruction implements the MicroExecute interface:

```

1 func (insn *Assign) MicroExecute(state State) (skip uint, pc uint) {
2   // Evaluate polynomial
3   value := insn.Source.Eval(state.Registers())
4   // Write to target registers
5   writeTargets(state, insn.Targets, value)
6   return 1, 0 // Continue to next code
7 }

```

Listing 10: Micro Execution Interface

Algorithm 3 Trace Propagation

```

1: function PROPAGATE(program, inputTrace)
2:   executor  $\leftarrow$  NewExecutor(program)
3:   WRITEINSTANCES(executor, inputTrace)
4:   for each function f in program do
5:     for each instance (inputs, outputs) of f do
6:       EXECUTE(f, inputs, executor.io)
7:     end for
8:   end for
9:   return READINSTANCES(executor)
10: end function

```

6.3 Trace Propagation

For function calls, the executor automatically propagates traces:

This ensures that if function f calls function g , all instances of g are automatically computed.

6.4 Instance Caching

The executor caches function instances to avoid redundant computation:

```

1 func (trace *ComponentTrace) Call(inputs []big.Int) Instance {
2   // Check cache
3   if cached := trace.instances.Find(inputs); cached != nil {
4     return cached
5   }
6   // Execute and cache
7   result := trace.execute(inputs)
8   trace.instances.Insert(result)
9   return result
10 }

```

Listing 11: Instance Caching

7 Integration with Constraint System

7.1 Architecture Overview

7.2 Mixed Program Model

The MixedProgram combines assembly and external modules:

```

1 type MixedProgram[F, T, M] struct {
2   program io.Program[T] // Assembly functions
3   externs []M           // External (Corset) modules
4 }
5
6 func (p *MixedProgram) Modules() Iterator[Module] {
7   // Concatenate assembly modules with external modules
8   return Chain(p.program.Modules(), p.externs)
9 }

```

Listing 12: MixedProgram Structure

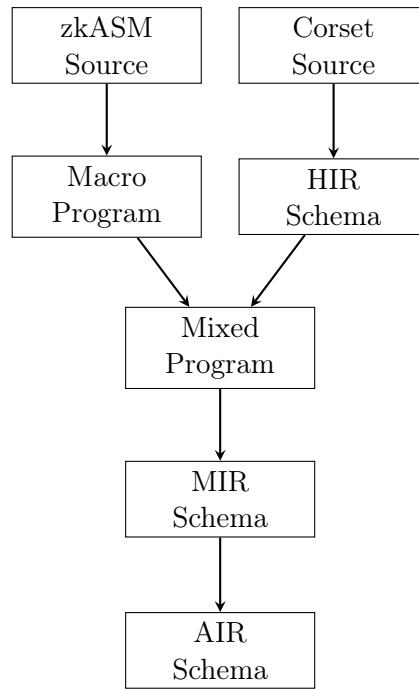


Figure 2: Integration Architecture

7.3 Module Interface

Each assembly function implements the schema module interface:

```

1 type Module[F] interface {
2     Name() string
3     Registers() []Register
4     Constraints() Iterator[Constraint[F]]
5     Assignments() Iterator[Assignment[F]]
6 }

```

Listing 13: Module Interface

7.4 Constraint Types

zkASM generates the following constraint types:

- **Vanishing Constraints:** Polynomial equations that must equal zero
- **Range Constraints:** Bit-width proofs for register values
- **Lookup Constraints:** Bus communication verification

8 Implementation Details

8.1 Package Structure

```

pkg/asm/
  assembler/          # Lexer, parser, linker, validator
    lexer.go
    parser.go
    linker.go

```

```

validation.go
compiler/      # Constraint generation
  compiler.go
  frame.go     # PC management
  insn_assign.go # Assignment compilation
  insn_ifgoto.go # Branch compilation
io/           # Core abstractions
  macro/      # Macro instructions
  micro/      # Micro instructions
  executor.go
  state.go
program/      # Module wrappers
assemble.go   # Pipeline coordinator

```

8.2 Key Data Structures

8.2.1 Register Representation

```

1 type Register struct {
2     Name      string
3     Width     uint    // Bit width
4     Padding   big.Int // Default/padding value
5     IsInput   bool
6     IsOutput  bool
7 }
8
9 type RegisterId struct {
10    Register uint    // Index in function's register list
11    Offset    int     // Row offset for shifted access
12 }

```

Listing 14: Register Definition

8.2.2 Instruction Representation

```

1 type Instruction struct {
2     Codes []Code // Sequence of micro codes
3 }
4
5 // Codes execute sequentially until skip=0
6 type Code interface {
7     MicroExecute(state State) (skip uint, pc uint)
8 }

```

Listing 15: Instruction Structure

8.3 Branch Table Analysis

For complex control flow, the compiler builds a branch table:

```

1 type BranchTable struct {
2     entries []BranchEntry
3 }
4
5 type BranchEntry struct {
6     condition Expression // Guard condition

```

```

7   target    uint           // Jump target
8 }
9
10 func BuildBranchTable(codes []Code, start uint) BranchTable {
11     // Traverse Skip chains to find all reachable targets
12     // Build condition for each path
13 }

```

Listing 16: Branch Table Construction

8.4 Frame Management

Two framing strategies support different function types:

One-Line Framing (simple functions):

- No program counter column needed
- All constraints active for every row
- Most efficient representation

Multi-Line Framing (functions with control flow):

- Program counter column tracks execution point
- Return flag indicates termination
- Constraints guarded by PC value

8.5 Performance Considerations

- **Vectorization:** Reduces constraint count by merging instructions
- **Instance Caching:** Avoids redundant function executions
- **Lazy Evaluation:** Constraints generated on-demand during compilation
- **Parallel Execution:** Thread-safe instance caching with mutex protection

9 Conclusion

zkASM provides a powerful intermediate representation for zero-knowledge circuit programming within the go-corset framework. Key contributions include:

1. A clean separation between high-level assembly semantics and low-level constraint generation
2. Field-agnostic programming through automatic register splitting
3. Efficient inter-function communication via the bus protocol
4. Optimization passes including vectorization and unreachable code elimination

The design enables developers to write portable zero-knowledge programs while the compiler handles field-specific adaptations automatically.

A zkASM Grammar

```

program      ::= declaration*
declaration  ::= constant | include | function

constant     ::= 'const' IDENTIFIER '=' NUMBER
include      ::= 'include' STRING
function     ::= 'fn' IDENTIFIER '(' params ')' '->' '(' returns ')' '{' body '}'

params       ::= param (',' param)*
param        ::= IDENTIFIER ('=' NUMBER)? type
returns      ::= param (',' param)*
type         ::= 'u' NUMBER

body         ::= (label | instruction)*
label        ::= IDENTIFIER ':'
instruction  ::= assignment | conditional | jump | return | call

assignment   ::= targets '=' expression
targets      ::= IDENTIFIER (',' IDENTIFIER)*
expression   ::= term (('+' | '-' | '*') term)*
term         ::= NUMBER | IDENTIFIER | '(' expression ')

conditional  ::= 'if' IDENTIFIER cmp_op expression 'goto' IDENTIFIER
cmp_op       ::= '==' | '!=' | '<' | '<=' | '>' | '>='

jump         ::= 'goto' IDENTIFIER
return       ::= 'return'
call         ::= targets '=' IDENTIFIER '(' arguments ')'
arguments    ::= expression (',' expression)*

```